Java Fundamentals & OOP

Classes, Objects, Encapsulation, Abstraction, Packages, Inheritance, Interfaces and Polymorphism - with live demos

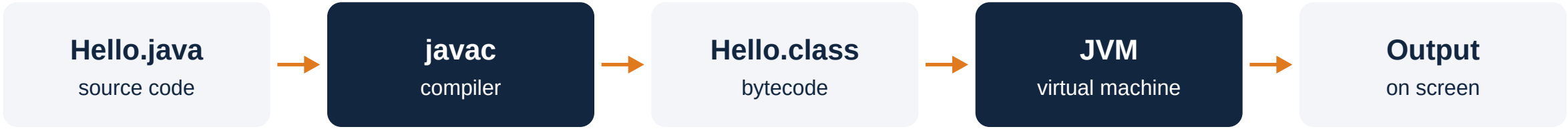
Agenda

- Java fundamentals
- Class and object
- OOP features: encapsulation, abstraction, inheritance, polymorphism
- Packages
- Abstract classes
- Interfaces: multiple, extending, default/static, functional + lambda
- Polymorphism: overloading (static) and reference types (runtime)
- Summary

What is Java?

- A programming language: a way to give instructions to a computer
- Created by James Gosling at Sun Microsystems; first released in 1995 (now Oracle)
- Used for banking systems, web applications, Android apps, enterprise software
- Why it is popular:
  - Simple, readable syntax and strong object-oriented design
  - Portable: the same program runs on Windows, Linux and Mac
  - Secure, huge standard library, very large community
- Java is case-sensitive: Account and account are different names

How a Java Program Runs



- Step 1 - You write human-readable code in a .java file
- Step 2 - javac compiles it into bytecode (.class file), checking syntax errors
- Step 3 - The JVM reads bytecode and runs it on your operating system
- Bytecode is the same everywhere; each OS has its own JVM = Write Once, Run Anywhere
- Commands: javac Hello.java then java Hello

JDK vs JRE vs JVM



Key points

- JVM runs the program
- JRE = JVM + libraries; enough to RUN Java apps
- JDK = JRE + compiler and tools; needed to WRITE Java apps
- Developers install the JDK

Your First Program - Anatomy

```
public class Hello {                          // class name = file name

    public static void main(String[] args) {  // program starts here

        System.out.println("Hello, Java!");   // print a line

    }
}
```

Output: `Hello, Java!`

Key points

- Every Java program lives inside a class
- main() is the entry point: the JVM calls it first
- System.out.println prints text and moves to a new line
- Every statement ends with ;
- { } mark the start and end of a block

Variables and Data Types

Type	Stores	Example
int	whole numbers	int age = 25;
double	decimal numbers	double balance = 5000.75;
char	one character	char grade = 'A';
boolean	true or false	boolean active = true;
String	text (a class, not a primitive)	String name = "Asha";

- A variable is a named box in memory that holds a value
- Rules: declare the type first, names start with a letter, no spaces

Java Fundamentals

- Platform-independent: write once, run anywhere (WORA)
- Source (.java) -> compiler (javac) -> bytecode (.class) -> JVM
- JDK = JRE + dev tools; JRE = JVM + libraries
- Strongly typed, object-oriented, automatic garbage collection
- Entry point: public static void main(String[] args)
- Primitives: int, double, boolean, char ... Objects: String, arrays, classes
- Java 5: varargs, enhanced for Java 8: lambda, default methods

Class and Object

- Class: blueprint - defines state (fields) and behaviour (methods)
- Object: a runtime instance of a class, created with new
- Constructor: special method that initialises an object (same name as class)
- Reference variable points to an object on the heap
-
- Account acc = new SavingAccount(101, "Asha", 5000, 4.5);
  - class = SavingAccount, object = acc, constructor call = new ...

Class vs Object - Real-Life Analogy

```
class Car {                             // blueprint
    String color;                       // state (fields)
    void drive() {                      // behaviour (method)
        System.out.println(color + " car is running");
    }
}
public class Main {
    public static void main(String[] args) {
        Car c1 = new Car();             // object 1
        c1.color = "Red";
        Car c2 = new Car();             // object 2
        c2.color = "Blue";
        c1.drive();
        c2.drive();
    }
}
```

Output:
```
Red car is running
Blue car is running
```

Key points

- Class = blueprint of a house; objects = actual houses built from it
- One class can create many objects
- Each object keeps its own copy of the fields
- new creates an object; the dot (.) accesses members

Anatomy of a Class

- A class has three main parts:
  - Fields (variables) - the data or STATE of the object: accountNo, name, balance
  - Constructors - set up the object when it is created with new
  - Methods - the BEHAVIOUR: what the object can do, e.g. withdraw()
-
- Special keywords you will meet:
  - new creates an object -this refers to the current object
  - static belongs to the class, not to one object
  - void method returns nothing -return sends a value back
-
- Naming: Classes = PascalCase (SavingAccount), methods/variables = camelCase (getBalance)

Constructors

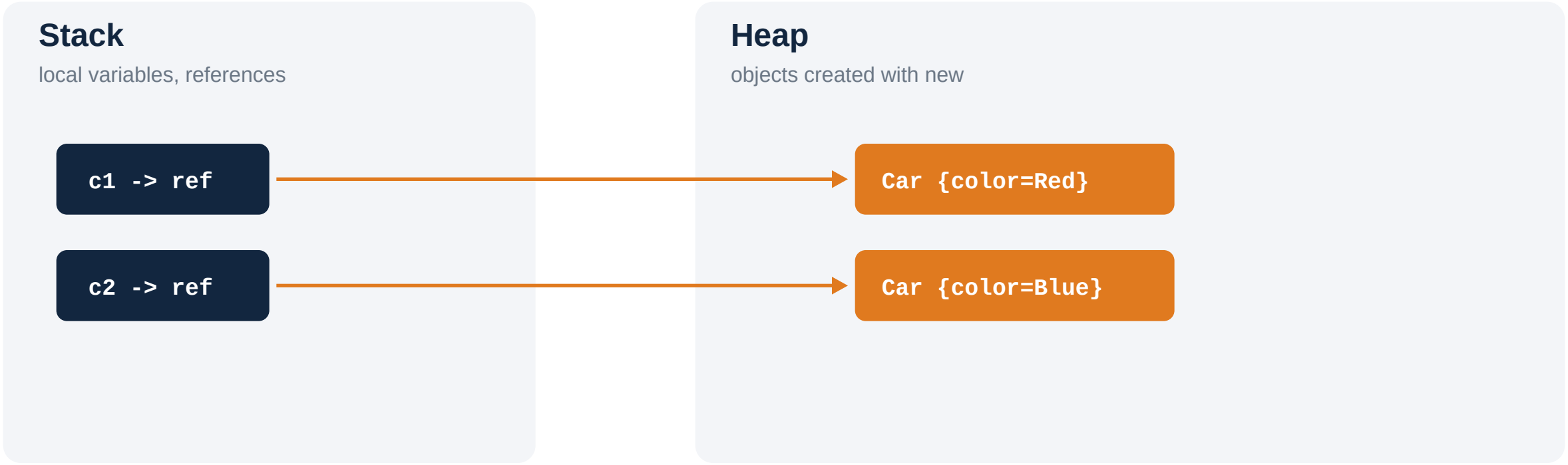
```
class Student {
    int id;
    String name;

    Student() {                          // default constructor
        id = 0;
        name = "Unknown";
    }
    Student(int id, String name) {       // parameterized constructor
        this.id = id;                    // this.id = field, id = parameter
        this.name = name;
    }
}
// new Student();              -> id=0, name=Unknown
// new Student(7, "Ravi");     -> id=7, name=Ravi
```

Key points

- Same name as the class, no return type
- Runs automatically when new is used
- If you write none, Java supplies an empty default one
- Several constructors = constructor overloading
- this separates field names from parameters

Where Objects Live: Stack and Heap



- The variable c1 holds a reference (address); the object itself is on the heap
- Objects nobody refers to any more are removed by the Garbage Collector

The Four Pillars of OOP

Encapsulation

Data hidden behind private fields + public getters/setters

Abstraction

Expose what an object does, hide how (abstract class, interfac e)

Inheritance

A subclass reuses and extends a superclass (extends)

Polymorphism

One interface, many forms (overloading, overriding)

Encapsulation - Theory

- Wrapping data and the methods that use it into ONE unit (class) and protecting the data
- Analogy: a medicine capsule, or an ATM - you use the screen, you cannot touch the cash
- How to achieve it:
  - 1. Declare fields private
  - 2. Provide public getters (read) and setters (write)
  - 3. Put validation inside the setters
- Benefits:
  - Data security - Control over changes - Easier maintenance - Flexibility

Encapsulation - Without vs With

```
// WITHOUT encapsulation (public field)
acc.balance = -5000;            // anyone can break the rule!

// WITH encapsulation (private field + setter)
private double balance;

public void setBalance(double balance) {
    if (balance >= 0) {
        this.balance = balance;
    } else {
        System.out.println("Invalid balance");
    }
}
// acc.setBalance(-5000);       -> Invalid balance
```

Key points

- The class protects its own data
- Validation lives in ONE place
- Outside code cannot bypass the rules

Access Modifiers

Modifier	Same class	Same package	Subclass	Everywhere
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

- Rule of thumb: fields private, methods public, and protected for subclass access
- In Account: accountNo and name are private; balance is protected so SavingAccount can use it

Encapsulation - Account (bean)

```
package com.usoe.bean;

public abstract class Account {
    private int accountNo;      // hidden from outside
    private String name;
    protected double balance;   // visible to subclasses

    public Account() { }
    public Account(int accountNo, String name, double balance) {
        this.accountNo = accountNo;
        this.name = name;
        this.balance = balance;
    }
    public int getAccountNo() { return accountNo; }
    public void setAccountNo(int accountNo) { this.accountNo = accountNo; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    // toString() prints accountNo, name, balance
}
```

Key points

- Fields are private: access only through methods
- Setters can validate before changing state
- this.x refers to the current object's field
- Constructors initialise the object; no-arg one kept for frameworks

Abstraction - Theory

- Show only WHAT an object does; hide HOW it does it
- Analogy: driving a car - you use steering and pedals, not the engine internals
- Analogy: ATM withdraw - you press a button, you do not see the bank's processing
- Achieved in Java using:
  - Abstract classes (partial abstraction) and interfaces (full abstraction)
- Abstract method = method declaration with NO body: public abstract void withdraw(double amt);
- Example: Account.withdraw() - Saving and Current accounts each withdraw differently

Abstraction - Abstract Class

```
public abstract class Account {
    ...
    // no body: every concrete account must define it
    public abstract void withdraw(double amount);
}

// Account a = new Account();   // ERROR: cannot instantiate

public class SavingAccount extends Account {
    private double interest;
    @Override
    public void withdraw(double amount) {
        balance = balance - amount;
        System.out.println("Remaining balance :- " + balance);
    }
}
```

Key points

- abstract class can mix concrete and abstract methods
- Cannot be instantiated with new
- Subclass must override all abstract methods or be abstract itself
- Use when classes share state and partial behaviour

Packages

- A package groups related classes and avoids name clashes
- Declared on the first line: package com.usoe.bean;
- Folder structure mirrors the package name
- Use import to access classes from other packages
-
- Used in this deck:
  - com.usoe.bean : Account, SavingAccount
  - com.usoe.util : Gst, TaxImpl
  - cg, cg1, cg2, cg3 : interface and lambda demos
- Access levels: private < default < protected < public

Packages - Using Them

- Built-in packages: java.lang (String, Math, System - imported automatically)
  - java.util (ArrayList, Scanner) java.io (files, streams)
- Naming convention: reverse domain, all lowercase: com.usoe.bean
- import examples:
  - import java.util.Scanner; import java.util.*;
- Benefits: organised code, no name clashes, access control, easy reuse
- Full name of a class includes the package: com.usoe.bean.Account
- Class from the same package needs no import

Inheritance - Theory

- A child class acquires the fields and methods of a parent class
- IS-A relationship: SavingAccount IS-A Account
- Terms: parent = superclass = base class; child = subclass = derived class
- Keyword: extends Keyword: super (access the parent)
- Benefits: code reuse, less duplication, easy to extend
- Example: Account -> SavingAccount, CurrentAccount
  - common data (accountNo, name, balance) written ONCE in Account
  - each child adds its own (interest, overdraft limit)
- Constructors are NOT inherited; the child calls the parent's with super(...)

Types of Inheritance

Type	Meaning	Example
Single	one parent, one child	Account -> SavingAccount
Multilevel	chain of classes	Account -> SavingAccount -> SeniorSaving
Hierarchical	one parent, many children	Account -> Saving, Current
Multiple	two parents for one class	NOT allowed with classes

- Why no multiple inheritance with classes? Diamond problem: if both parents have the same method, which one runs?
- Java solves this with interfaces: a class can implement many of them

Inheritance - SavingAccount extends Account

```
package com.usoe.bean;

public class SavingAccount extends Account {
    private double interest;

    public SavingAccount() { }
    public SavingAccount(int accountNo, String name,
                         double balance, double interest) {
        super(accountNo, name, balance);   // call parent constructor
        this.interest = interest;
    }
    public double getInterest() { return interest; }
    public void setInterest(double interest) { this.interest = interest; }

    @Override
    public String toString() {
        return "SavingAccount [interest=" + interest + "]" + super.toString();
    }
}
```

Key points

- extends gives IS-A relationship: SavingAccount IS-A Account
- super(...) must be the first statement in the constructor
- super.toString() reuses parent logic
- Java supports single class inheritance only

Method Overriding - Rules

- Child class gives its own version of a method inherited from the parent
- Method name, parameters and return type must match the parent
- Access cannot be reduced (public parent method -> public in child)
- Always add @Override: the compiler catches spelling or signature mistakes
- private, static and final methods cannot be overridden
- super.method() calls the parent version
- Example: SavingAccount overrides Account's abstract withdraw()
- Overriding = runtime polymorphism: the object's real class decides which method runs

Interface - Theory

- A contract: a list of methods a class promises to provide
- Analogy: a USB port or a TV remote - same buttons, different devices behind them
- Rules:
  - Methods are public abstract by default; fields are public static final
  - No objects can be created from an interface
  - A class uses implements and must define every method
  - A class can implement MANY interfaces; an interface can extend MANY interfaces
- Java 8 added default and static methods inside interfaces

Interface - Gst and TaxImpl

```
package com.usoe.util;

public interface Gst {
    public double FIVEPCT = 0.05;   // implicitly public static final
    public double ONEPCT  = 0.01;

    public void calculateTax(double amount, double pct);  // implicitly abstract
}

public class TaxImpl implements Gst {
    @Override
    public void calculateTax(double amount, double pct) {
        System.out.println("Tax is " + (amount * pct));
    }
}

// Gst g = new TaxImpl();  g.calculateTax(1000, Gst.FIVEPCT);   // Tax is 50.0
```

Key points

- Interface = contract: what, not how
- Fields are constants; methods are abstract by default
- Class uses implements and must define every method
- 100% abstraction (before Java 8 default methods)

Multiple Interfaces (Demo1)

```
interface Shape   { public void area(); }
interface Shape3d { public void volume(); }

class Cube implements Shape, Shape3d {      // multiple interfaces
    double side;
    public void volume() { System.out.println("Volume is Cube " + side*side*side); }
    public void area()   { System.out.println("Area of Cube " + 3*(side*side)); }
}

public class Demo1 {
    public static void main(String[] args) {
        Cube ob = new Cube();  ob.side = 7.00;
        ob.area();  ob.volume();
        Shape s = ob;      s.area();
        //s.volume();      // error: Shape has no volume()
        Shape3d sd = ob;   sd.volume();
        //sd.area();       // error: Shape3d has no area()
    }
}
```

Key points

- A class can implement many interfaces: Java's answer to multiple inheritance
- Reference type decides which methods are callable
- Object is the same Cube in all three variables

Output: `Area of Cube 147.0 | Volume is Cube 343.0 | Area of Cube 147.0 | Volume is Cube 343.0`

Interface Extends Interface (Demo2)

```
interface Shape { public void area(); }

interface Shape3d extends Shape {          // sub-interface
    public void volume();
}

// Cube must override super AND sub interface methods
class Cube implements Shape3d {
    double side;
    public void volume() { ... }
    public void area()   { ... }
}

public class Demo2 {
    public static void main(String[] args) {
        Cube ob = new Cube();  ob.side = 7.00;
        Shape s = ob;      s.area();
        Shape3d sd = ob;   sd.volume();
        sd.area();         // allowed: inherited from Shape
    }
}
```

Key points

- interface extends interface (can extend several)
- Implementing class must define all inherited methods
- Shape3d reference can call both area() and volume()

Java 8: Default and Static Methods

```
package cg2;

public interface Conversion {
    public void show();                         // abstract

    public default double calculateTax(double amount) {   // default
        return amount * 0.02;
    }
    public static double dollarToRs(double dallar) {      // static
        return dallar * 75.00;
    }
}

public class ConversionImpl implements Conversion {
    public void show() { System.out.println("Abstract method -- show()"); }
    // overriding calculateTax() is optional
}

// Demo: c.show();  c.calculateTax(10000.00);  Conversion.dollarToRs(100);
```

Key points

- default: common code, optional to override
- static: utility, called on the interface name
- Lets interfaces evolve without breaking implementers

Output:

```
Abstract method -- show()
Tax is 200.0
100 dollars = 7500.0 Rs.
```

Functional Interface and Lambda (Demo3)

```
@FunctionalInterface
interface Tax {
    public void caltax(double amount);       // single abstract method (SAM)
}

public class Demo3 {
    public static void main(String[] args) {
        Tax ob1 = (double amount) -> {
            System.out.println("Tax is " + (0.20 * amount));
        };
        // Tax ob1 = (amount) -> { ... };    // data type optional
        // Tax ob1 = amount -> { ... };      // brackets optional (1 param)
        ob1.caltax(10000.00);
    }
}
```

Key points

- Exactly one abstract method = functional interface
- Lambda replaces an anonymous inner class
- (parameters) -> { body }

Output: `Tax is 2000.0`

Abstract Class vs Interface

Feature	Abstract class	Interface
Keyword	abstract class	interface
Methods	abstract + normal	abstract (+ default, static)
Variables	any kind	constants only
Constructor	yes	no
Inheritance	extends ONE class	implements MANY
Use when	shared state + behaviour	common capability / contract

Polymorphism

- Poly = many, morph = forms: one name, different behaviours
-
- Compile-time (static): method overloading
  - same name, different parameter type / number / order
  - resolved by the compiler
-
- Run-time (dynamic): method overriding
  - subclass redefines a parent / interface method
  - resolved by the JVM using the actual object
-
- Reference of interface or parent type can hold any child object

Polymorphism - Analogy

- The same person is a son at home, a student in class, an employee at work
- The word 'move' means different things for a car, a bird and a fish
- In Java: ONE method name or reference, MANY behaviours
- Overloading: same method name, different parameters, in the SAME class
- Overriding: child class replaces a parent's method
-
- Example in our demos:
  - add("10","12") and add(10,12) - overloading in Operation
  - Shape s = new Cube(); s.area(); - the Cube's area() runs (overriding)

Overloading vs Overriding

Feature	Overloading	Overriding
Where	same class	parent and child
Method signature	must differ	must be the same
Return type	can differ	same (or subtype)
Resolved	compile time	run time
Also called	static polymorphism	dynamic polymorphism

Method Overloading - Operation

```
public class Operation {
    public void add(String a, String b)  { System.out.println(a + b); }
    public void add(String a, double b)  { System.out.println(a + b); }
    public void add(double a, String b)  { System.out.println(a + b); }

    // variable arguments (JDK 1.5+): n double values
    public void add(double... a) {
        System.out.println("Variable arguments method");
        double sum = 0.0;
        for (double d : a) sum += d;       // enhanced for loop
        System.out.println(sum);
    }
    // varargs must be the LAST parameter
    public void add(String s, double... a) {
        double sum = 0.0;
        System.out.println(s);
        for (double d : a) sum += d;
        System.out.println(sum);
    }
}
```

Key points

- Same name, different signature
- Return type alone cannot overload
- double... behaves like double[]
- Varargs always last

Overloading - Demo

```
public class Demo {
    public static void main(String[] args) {
        Operation ob = new Operation();
        ob.add("10", "12");                 // add(String, String)
        ob.add("10", 12);                   // add(String, double)
        ob.add(10, "12");                   // add(double, String)
        ob.add(10.67, 12.11);               // add(double...)
        ob.add(10.67, 12.11, 22.55);        // add(double...)
        double x[] = {34.565, 32.989, 767.878, 11.11, 99.34};
        ob.add(x);                          // array passed to varargs
        ob.add(44.0, 66.0, 77.0, 22.0, 66.0, 88.0, 99.0, 55.0);
        ob.add("Different ", 10, 20, .0, 30.5);   // add(String, double...)
    }
}
```

Key points

- Compiler picks the best-matching signature
- int 10 widens to double 10.0
- Any count of doubles fits add(double...)

Output: `1012 | 1012.0 | 10.012 | (varargs: sums) | ... | Different  then 60.5`

Runtime Polymorphism - Reference Types

- Parent or interface reference can point to a child object
-
- Shape s = ob; // Shape -> Cube
- Shape3d sd = ob; // Shape3d -> Cube
- Conversion c = new ConversionImpl();
- Account a = new SavingAccount(101, "Asha", 5000, 4.5);
-
- a.withdraw(500); // JVM runs SavingAccount.withdraw()
  - compile time: only methods of the reference type are visible
  - run time: the actual object's overridden method executes

Common Beginner Mistakes

- Public class name must match the file name (Hello.java -> class Hello)
- Missing semicolon ; or an unclosed brace { }
- Using == to compare Strings: use equals()
- Forgetting new: the reference stays null -> NullPointerException
- Trying to create an object of an abstract class or interface
- Not overriding every abstract method in the child class
- Integer division: 5 / 2 gives 2, not 2.5 (use 5 / 2.0)
- Case mistakes: string vs String, system vs System

Practice Questions

- 1. What is the difference between a class and an object?
- 2. Why do we make fields private? What are getters and setters for?
- 3. What does super(...) do inside a constructor?
- 4. Can we create an object of an abstract class? Can it have a constructor?
- 5. Give two differences between an abstract class and an interface.
- 6. Which kind of polymorphism is method overloading? Why?
- 7. What happens if a child class does not override an abstract method?
- 8. Write a Student class with a parameterized constructor and a display() method.

Summary

- Class = blueprint, object = instance
- Encapsulation: private data + getters/setters (Account)
- Abstraction: abstract class and interface (withdraw, Gst)
- Inheritance: extends reuses code (SavingAccount)
- Interfaces: multiple, extending, default/static, functional
- Polymorphism: overloading (static) and overriding (runtime)
- Packages organise code; Java 8 adds lambda expressions